SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OPERATING SYSTEMS **COURSE CODE: CSA04**

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance

QUESTIONS: 5 **Total Marks: 100**

Annexure A

Analytical Problem Solving

Code of Conduct:

I M. Pranay (192525382)_ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *M. Pranay*

QUESTIONS

Question 1: CPU Scheduling – Waiting Time & Turnaround Time Analysis

A set of processes arrive at time 0 with the following burst times:

Process	Burst Time (ms)
P1	10
P2	4
P3	6
P4	2

a. Calculate Waiting Time and Turnaround Time for each process using SJF (Non-preemptive).
b. Compute the average waiting time and average turnaround time.
c. Interpret whether SJF improves CPU utilization compared to FCFS for this workload.

(20 Marks) (BL4)

Question 2: Round Robin Scheduling – Time Quantum Impact

Consider the following processes:

Process	Burst Time (ms)
P1	20
P2	5
P3	13
P4	8

Let the time quantum = 4 ms.

a. Construct the Gantt chart for Round Robin.
b. Calculate Waiting Time and Turnaround Time of each process.
c. Analyze how changing the quantum to 10 ms would affect system performance (CPU utilization & context switching).

(20 Marks) (BL4)

Question 3: Deadlock – Safety Check Using Banker's Algorithm

A system has 3 resource types (A, B, C). Total resources:
A = 10, B = 5, C = 7.

The Allocation and Maximum Need matrices are:

Allocation Matrix

Process	A	B	C
P1	0	1	0
P2	2	0	0
P3	3	0	2
P4	2	1	1
P5	0	0	2

Maximum Matrix

Process	A	B	C
P1	7	5	3
P2	3	2	2
P3	9	0	2
P4	4	2	2
P5	5	3	3

a. Calculate the Need Matrix.
b. Determine if the system is in a safe state using Banker's Algorithm.
c. Provide a valid safe sequence if it exists.

(20 Marks) (BL4)

Question 4: Deadlock Detection – Resource Allocation Graph (RAG)

A system has the following resource allocations:

- P1 is holding R1 and waiting for R2

- P2 is holding R2 and waiting for R3
- P3 is holding R3 and waiting for R1

a. Draw the Resource Allocation Graph (describe in text).
b. Determine whether the system is in deadlock.
c. Recommend a strategy (prevention OR avoidance OR recovery) to resolve it and justify the method.

(20 Marks) (BL4)

Question 5: CPU Utilization – Multiprogramming Level Analysis

A system has the following process behavior:

- Each process spends 20% time in CPU and 80% waiting for I/O.
- Degree of multiprogramming = 4 processes

CPU Utilization formula:

where:

- p = probability a process is waiting for I/O,
- n = number of processes in memory

a. Calculate CPU Utilization.
b. Compute CPU utilization if the degree of multiprogramming increases to 6.
c. Interpret which configuration improves system performance and why.

(20 Marks) (BL4)

RUBRICS FOR CALCULATION
Analytical Problems

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

TOTAL MARKS AWARDED **FACULTY SIGNATURE**

ANSWERS

1. CPU Scheduling – Waiting Time and Turnaround Time

Given,

Burst times: P1 = 10 ms, P2 = 4 ms, P3 = 6 ms, P4 = 2 ms. All processes arrive at time 0. Use non-preemptive Shortest Job First (SJF).

a) SJF order and Gantt chart

Arrange processes in increasing order of burst time: P4 → P2 → P3 → P1.

Gantt chart: 0 | P4 | 2 | P2 | 6 | P3 | 12 | P1 | 22

b) Waiting time and turnaround time

Waiting Time (WT) = Start Time − Arrival Time. Turnaround Time (TAT) = Completion Time − Arrival Time.

Process	Burst (ms)	Start (ms)	Completion (ms)	WT (ms)	TAT (ms)
P1	10	12	22	12	22
P2	4	2	6	2	6
P3	6	6	12	6	12
P4	2	0	2	0	2
Total	22			20	42

Average WT = (12 + 2 + 6 + 0) / 4 = 5 ms.

Average TAT = (22 + 6 + 12 + 2) / 4 = 10.5 ms.

c) Comparison with FCFS

Algorithm	Average WT (ms)	Average TAT (ms)
FCFS	11	16.5
SJF	5	10.5

SJF reduces average waiting time and average turnaround time by 6 ms compared with FCFS. CPU utilization does not improve in this workload: all processes arrive at time 0, so the CPU remains busy for 22 ms under either algorithm, with no idle time.

2. Round Robin Scheduling – Time Quantum Impact

Burst times: P1 = 20 ms, P2 = 5 ms, P3 = 13 ms, P4 = 8 ms. Time quantum = 4 ms. Assume all arrive at time 0 and the ready-queue order is P1, P2, P3, P4.

a) Gantt chart

0–4 P1 | 4–8 P2 | 8–12 P3 | 12–16 P4 | 16–20 P1 | 20–21 P2 | 21–25 P3 | 25–29 P4 | 29–33 P1 | 33–37 P3 | 37–41 P1 | 41–42 P3 | 42–46 P1

b) Waiting time and turnaround time

TAT = Completion Time − Arrival Time. WT = TAT − Burst Time. Since all arrival times are zero, TAT equals completion time.

Process	Burst (ms)	Completion (ms)	TAT (ms)	WT (ms)
P1	20	46	46	26
P2	5	21	21	16
P3	13	42	42	29
P4	8	29	29	21
Total	46		138	92

Average WT = 92 / 4 = 23 ms.
Average TAT = 138 / 4 = 34.5 ms.

c) Effect of increasing quantum to 10 ms

- A larger quantum generally reduces context-switching frequency and scheduling overhead.
- Interactive or short processes may wait longer for their next turn, so responsiveness may decrease.
- CPU utilization should remain high when there is always a ready process; changing the quantum mainly affects overhead and response behavior, not the total CPU burst requirement.

3. Deadlock – Safety Check Using Banker's Algorithm

Total resources: A = 10, B = 5, C = 7.

a) Need matrix

Need = Maximum − Allocation.

Process	Allocation (A,B,C)	Maximum (A,B,C)	Need (A,B,C)
P1	(0,1,0)	(7,5,3)	(7,4,3)
P2	(2,0,0)	(3,2,2)	(1,2,2)
P3	(3,0,2)	(9,0,2)	(6,0,0)
P4	(2,1,1)	(4,2,2)	(2,1,1)
P5	(0,0,2)	(5,3,3)	(5,3,1)

b) Available resources and safety check

Allocated totals: A = 7, B = 2, C = 5. Therefore, Available = Total − Allocated = (10−7, 5−2, 7−5) = (3,3,2).

Step	Process	Need	Available before	Available after completion
1	P2	(1,2,2)	(3,3,2)	(5,3,2)
2	P4	(2,1,1)	(5,3,2)	(7,4,3)
3	P1	(7,4,3)	(7,4,3)	(7,5,3)
4	P3	(6,0,0)	(7,5,3)	(10,5,5)
5	P5	(5,3,1)	(10,5,5)	(10,5,7)

At each step, the selected process has Need ≤ Available. When it finishes, its allocated resources are returned to the available pool.

c) Safe sequence and conclusion

A valid safe sequence is P2 → P4 → P1 → P3 → P5. Therefore, the system is in a safe state because every process can finish in this order without deadlock.

4. Deadlock Detection – Resource Allocation Graph

a) Graph description

Allocation edges: R1 → P1, R2 → P2, R3 → P3. Request edges: P1 → R2, P2 → R3, P3 → R1.

The graph forms the cycle: P1 → R2 → P2 → R3 → P3 → R1 → P1.

b) Determine whether deadlock exists

Yes. Assuming each resource has a single instance, P1 holds R1 and waits for R2, P2 holds R2 and waits for R3, and P3 holds R3 and waits for R1. Each process is waiting for a resource held by another process in the cycle, so none can proceed.

c) Recommended strategy: prevention

Use resource ordering. Define a fixed order R1 < R2 < R3 and require processes to request resources only in that order. A process needing R1 and R3 must request R1 before R3. This prevents circular wait and thereby prevents this form of deadlock.

5. CPU Utilization – Multiprogramming Level Analysis

Each process spends 80% of its time waiting for I/O, so p = 0.8. The formula is $U = 1 - p^n$, where n is the number of processes in memory.

a) Four processes

$U = 1 - (0.8)^4 = 1 - 0.4096 = 0.5904 = 59.04\%$.

b) Six processes

$U = 1 - (0.8)^6 = 1 - 0.262144 = 0.737856 \approx 73.79\%$.

c) Interpretation

Degree of multiprogramming	CPU utilization
4 processes	59.04%
6 processes	73.79%

Increasing the number of processes from 4 to 6 improves the calculated CPU utilization by approximately 14.75 percentage points. While one process waits for I/O, another can use the CPU, reducing idle time. However, excessive multiprogramming can cause memory pressure and scheduling overhead, so the level should match available system resources.